1. What is CSS Box Model?

Every HTML element is treated as a rectangular box made of four layers, from inside out:

- Content – the actual text/image, sized by width and height
- Padding – transparent space between content and border
- Border – a line surrounding the padding
- Margin – transparent space outside the border, separating it from other elements

```
div {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
```

By default, width/height apply only to content (box-sizing: content-box), so total width = width + padding + border. Setting box-sizing: border-box makes width/height include padding and border, which is why many developers add:

```
* { box-sizing: border-box; }
```

2. What is Media Query? Explain with a sample code.

A media query applies CSS rules only when certain conditions (device width, orientation, resolution, etc.) are met — the foundation of responsive design.

```
/* Default styles for mobile */
body {
  font-size: 14px;
}

/* Styles for tablets and larger */
@media (min-width: 768px) {
  body {
    font-size: 16px;
  }
}

/* Styles for desktops */
@media (min-width: 1024px) {
  body {
    font-size: 18px;
  }
}
```

3. What are the features of Bootstrap?

- Responsive grid system – 12-column layout that adapts to screen size
- Pre-styled components – navbars, buttons, cards, modals, carousels, forms
- Mobile-first design – built to scale down from desktop to mobile smoothly
- Cross-browser compatibility – works consistently across major browsers

- Utility classes – spacing, colors, flexbox, display helpers without writing custom CSS
- JavaScript plugins – dropdowns, tooltips, modals, carousels (via Bootstrap JS/Popper)
- Customizable – can override via Sass variables or custom themes
- Consistent typography and icons – uniform look across a project
- Large community/documentation – easy to learn and get support

4. Write an example of basic grid structure in Bootstrap

```
<div class="container">
  <div class="row">
    <div class="col-md-4">Column 1</div>
    <div class="col-md-4">Column 2</div>
    <div class="col-md-4">Column 3</div>
  </div>
</div>
```

- container – wraps content with fixed/responsive width
- row – a horizontal group of columns (uses flexbox)
- col-md-4 – each column takes 4 of 12 grid units on medium+ screens (3 equal columns)

5. What is CSS Selectors?

Selectors are patterns used to target HTML elements for styling. Common types:

Selector	Example	Selects
Universal	*	all elements
Type/Element	p	all <p> tags
Class	.box	elements with class="box"
ID	#header	element with id="header"
Descendant	div p	<p> inside
Child	div > p	direct <p> children of
Attribute	input[type="text"]	inputs with that attribute value
Pseudo-class	a:hover	link on hover
Pseudo-element	p::first-line	first line of a paragraph

6. What are CSS Backgrounds? List the properties.

CSS lets you style the background of an element with these properties:

- background-color – sets a solid color
- background-image – sets an image (url(...))
- background-repeat – controls tiling (repeat, no-repeat, repeat-x, repeat-y)
- background-position – positions the image (center, top left, 50% 50%, etc.)
- background-size – scales the image (cover, contain, or specific dimensions)
- background-attachment – scroll behavior (scroll, fixed, local)
- background-origin – defines positioning area (border-box, padding-box, content-box)
- background-clip – how far background extends (border-box, padding-box, content-box, text)
- background – shorthand combining all the above

```
div {
  background: url('image.jpg') no-repeat center/cover fixed;
}
```

7. How can we overlap elements in CSS?

Use positioning with z-index:

```
.box1 {
  position: absolute;
  top: 20px;
  left: 20px;
  z-index: 1;
}

.box2 {
  position: absolute;
  top: 40px;
  left: 40px;
  z-index: 2; /* appears above box1 */
}
```

Key points:

- Elements need position: relative, absolute, fixed, or sticky for z-index to take effect
- Higher z-index value = closer to viewer (renders on top)
- A parent with negative/auto margins, or grid-template-areas placing items in the same cell, can also cause overlap

8. What are the different positioning properties in CSS?

- **static** – default; follows normal document flow, top/left etc. have no effect
- **relative** – positioned relative to its normal position; offsets (top, left) shift it without removing it from flow
- **absolute** – removed from normal flow; positioned relative to the nearest positioned ancestor (or the document if none)
- **fixed** – positioned relative to the browser viewport; stays fixed even when scrolling
- **sticky** – toggles between relative and fixed depending on scroll position (e.g., sticks to top after scrolling past it)

```
.sticky-nav {
  position: sticky;
  top: 0;
}
```

9. What is ECMAScript?

ECMAScript (ES) is the standardized specification that JavaScript is based on, maintained by ECMA International (TC39 committee). JavaScript is essentially an implementation of the ECMAScript standard (other implementations include JScript and ActionScript).

Key versions:

- ES5 (2009) – widely supported baseline

- ES6 / ES2015 – major update: let/const, arrow functions, classes, template literals, promises, modules
- ES2016 and later – yearly incremental updates (ES2017, ES2020, etc.) adding features like async/await, optional chaining, BigInt

In short: **JavaScript = ECMAScript + additional browser/runtime APIs** (like DOM, fetch, setTimeout).

10. Difference between var, let and const in JavaScript. Explain Temporal Dead Zone.

Feature	var	let	const
Scope	Function-scoped	Block-scoped	Block-scoped
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed
Hoisting	Hoisted & initialized as undefined	Hoisted but not initialized	Hoisted but not initialized

```
function example() {
  if (true) {
    var a = 10;   // function-scoped
    let b = 20;   // block-scoped
    const c = 30; // block-scoped, immutable binding
  }
  console.log(a); // 10 (accessible)
  console.log(b); // ReferenceError (not accessible outside block)
}
```

Temporal Dead Zone (TDZ):

The TDZ is the period between entering a scope and the point where a let/const variable is actually declared. During this time, the variable exists (it's hoisted) but cannot be accessed — doing so throws a ReferenceError.

```
console.log(x); // ReferenceError: Cannot access 'x' before initialization
let x = 5;
```

Unlike var (which would just print undefined), let/const block any access before their declaration line is executed — this protects against using variables before they're properly initialized.